



Plano de desenvolvimento

Tabela de apoio

A tabela abaixo contempla vários recursos das bibliotecas que vamos usar para escrever o código do nosso EP. Consulte-a como ponto de partida.

Biblioteca / classe	Função	O que faz na prática	Exemplo de uso
<code>socket</code>	<code>socket(AF_INET, SOCK_STREAM)</code>	Instancia o socket TCP. Focado em conexões confiáveis e ordenadas.	<code>s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)</code>
<code>socket</code>	<code>socket(AF_INET, SOCK_DGRAM)</code>	Instancia o socket UDP. Focado em alta velocidade e sem garantias de entrega, ideal para o P2P.	<code>s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)</code>
<code>socket</code>	<code>bind((ip, porta))</code>	(Servidor) Associa o socket a um endereço IP e porta física específicos da máquina.	<code>s.bind(('127.0.0.1', 5050))</code>
<code>socket</code>	<code>listen()</code>	(Servidor TCP) Coloca o socket na porta especificada aguardando conexões.	<code>s.listen()</code>
<code>socket</code>	<code>accept()</code>	(Servidor TCP - bloqueante) Trava a execução até um cliente bater na porta. Retorna um novo socket exclusivo e o IP do cliente.	<code>conn, addr = s.accept()</code>
<code>socket</code>	<code>connect((ip, porta))</code>	(Cliente TCP) Inicia a tentativa	<code>s.connect(('127.0.0.1', 5050))</code>

		de conexão com o servidor.	
socket	sendall(bytes)	(TCP) Garante que toda a mensagem seja empurrada para a rede através do canal aberto.	conn.sendall(b"AUTH_CONN Sabrina\n")
socket	recv(tamanho)	(TCP - Bloqueante) Lê os pacotes de dados no buffer. Retorna vazio (b"") se a conexão cair.	dados = conn.recv(1024)
socket	sendto(bytes, (ip, p))	(UDP) Atira a mensagem diretamente para o destino, sem necessidade de conexão prévia. Usado no minigame.	s.sendto(b"GAME_ACTION", (ip_peer, porta_peer))
socket	recvfrom(tamanho)	(UDP - Bloqueante) Recebe o pacote e descobre instantaneamente de quem veio.	dados, addr = s.recvfrom(1024)
socket	close()	Fecha a conexão, quebra o canal e libera a porta no sistema operacional.	s.close()
threading	Thread(target, args)	Prepara uma nova linha de execução paralela, apontando para uma função do seu código.	t = threading.Thread(target=atender_cliente, args=(conn,))
threading	start()	Dá a ordem para a thread começar a executar a função simultaneamente ao resto do código.	t.start()

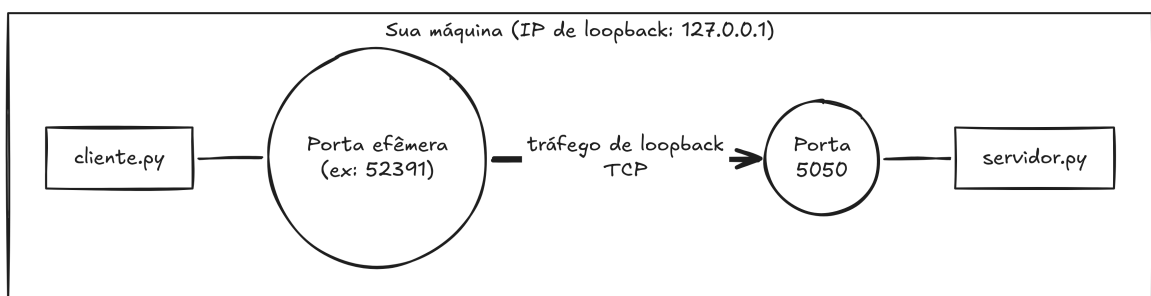
<code>str</code> (nativo)	<code>encode('utf-8')</code>	Traduz o seu texto legível em bytes para trafegar pela rede.	<code>"ECHO_REPLY".encode('utf-8')</code>
<code>bytes</code> (nativo)	<code>decode('utf-8')</code>	Pega a sequência confusa de bytes que veio da rede e transforma de volta em string.	<code>texto_limpo = dados.decode('utf-8')</code>

Fase 1: fundação e protocolo

Objetivo: Provar que um cliente e um servidor Python se conectam via TCP e conseguem se entender usando o formato COMANDO PAYLOAD.

✓ Issue 1: estruturação inicial do repositório e constantes de rede

- **Descrição:** Criar a estrutura base de pastas e arquivos conforme o planejamento arquitetural do projeto, além de definir um arquivo ou módulo de configurações compartilhadas para a rede.
- **Tarefas:**
 1. Criar os diretórios `servidor/`, `cliente/` e `docs/`.
 2. Criar os arquivos vazios `servidor/servidor.py`, `cliente/rede.py` e `cliente/main.py`.
 3. Definir as constantes padrões de rede no arquivo `config.py` na raiz do projeto: `HOST = '127.0.0.1'` (localhost), `PORT = 5050`, `BUFFER_SIZE = 1024` e `ENCODING = 'utf-8'`.



✓ Issue 2: criação do socket TCP passivo no servidor

- **Descrição:** Configurar o `servidor.py` para instanciar um socket TCP, associá-lo a um endereço/porta e colocá-lo em modo de escuta para aguardar conexões.
- **Tarefas:**
 1. Instanciar o socket utilizando `socket.socket(socket.AF_INET, socket.SOCK_STREAM)` (IPv4 + TCP).
 2. Adicionar a opção `SO_REUSEADDR` no socket para evitar erro de porta ocupada ao reiniciar o servidor rapidamente (`socket.setsockopt`).

3. Realizar o `bind((HOST, PORT))` e chamar o método `listen()`.
4. Adicionar um log no terminal indicando: `[SERVIDOR] Aguardando conexões na porta 5050...`.

✓ ~~Issue 3: criação do socket TCP ativo no cliente e conexão~~

- **Descrição:** Implementar a lógica inicial em `cliente/rede.py` para criar um socket TCP e conectar ao servidor.
- **Tarefas:**
 1. Instanciar o socket do cliente com `socket.AF_INET` e `socket.SOCK_STREAM`.
 2. Implementar a função `conectar_servidor(host, port)` que executa `client_socket.connect((host, port))`.
 3. No `servidor.py`, adicionar a chamada bloqueante `conn, addr = server_socket.accept()` e exibir uma mensagem informando o IP e porta efêmera do cliente conectado.

✓ ~~Issue 4: módulo de empacotamento e parsing do protocolo~~

- **Descrição:** Implementar funções utilitárias para formatar mensagens antes do envio e interpretá-las na recepção, seguindo o padrão textual delimitado.
- **Tarefas:**
 1. Definir o delimitador de fim de mensagem (ex: caractere de quebra de linha `\n`) para que o receptor saiba onde termina uma mensagem.
 2. Criar a função `formatar_mensagem(comando: str, payload: str = "") -> bytes`, retornando a string `"{comando} {payload}\n"` codificada em `utf-8`.
 3. Criar a função `interpretar_mensagem(dados_brutos: str) -> tuple[str, str]`, que separa a string no primeiro espaço, retornando o `COMANDO` e o `PAYLOAD`.

✓ ~~Issue 5: implementação do fluxo echo (envio, recepção e resposta)~~

- **Descrição:** Fazer o cliente enviar um comando `ECHO` com um texto de teste, o servidor receber, interpretar, responder com `ECHO_REPLY` e o cliente exibir a resposta no terminal.
- **Tarefas:**
 1. No cliente, após conectar, enviar uma mensagem usando `client_socket.sendall(formatar_mensagem("ECHO", "Teste de conexao"))`.
 2. No servidor, ler os dados com `dados = conn.recv(1024).decode('utf-8')`.
 3. No servidor, interpretar o comando: se for `ECHO`, responder ao cliente com `conn.sendall(formatar_mensagem("ECHO_REPLY", payload))`.
 4. No cliente, aguardar a resposta com `recv()`, interpretar a mensagem e imprimir: `[RESPOSTA RECEBIDA]: ECHO_REPLY -> Teste de conexao`.

✓ ~~Issue 6: encerramento de conexão e tratamento de erros~~

- **Descrição:** Garantir que o fechamento da conexão (voluntário ou por erro) seja tratado sem quebrar o servidor ou deixar portas presas.
- **Tarefas:**
 1. Envolver as rotinas de rede do cliente e do servidor em blocos `try...finally`.
 2. Adicionar o fechamento explícito dos sockets com `.close()` no `finally`.
 3. Tratar a desconexão no servidor: quando `conn.recv()` retornar `b""` (zero bytes), reconhecer que o cliente encerrou a conexão e fechar o socket do lado do servidor sem gerar exceção `ConnectionResetError`.

Fase 2: autenticação e gerenciamento de clientes

Objetivo: O servidor agora gerencia múltiplos clientes simultaneamente (threads), e o cliente consegue se logar no servidor (AUTH_CONN), sendo adicionado a uma lista online no servidor.

✓ Issue 7: threading no servidor

- **Descrição:** Modificar o comportamento do `servidor.py` para que ele não pare no primeiro cliente. Ele deve rodar um loop contínuo de `accept()` (a recepcionista) e, para cada conexão recebida, abrir uma nova linha de execução (o garçom) usando a biblioteca `threading`.
- **Tarefas:**
 1. Importar a biblioteca nativa `import threading`.
 2. Criar uma função chamada `lidar_com_cliente(conexao, endereco)`, e mover toda a lógica de `recv()` e `sendall()` (que fizemos na Issue 5) para dentro dela.
 3. No bloco principal do servidor, envolver a chamada `conexao, endereco = server_socket.accept()` em um laço `while True:`.
 4. Logo após o `accept()`, instanciar a thread: `thread = threading.Thread(target=lidar_com_cliente, args=(conexao, endereco))`.
 5. Iniciar a thread com `thread.start()`.

✓ Issue 8: estrutura de estado global

- **Descrição:** O servidor precisa saber quem está no lobby principal. Precisamos de uma estrutura de dados compartilhada para mapear o socket de cada cliente ao seu respectivo nome (nickname).
- **Tarefas:**
 1. No escopo global do `servidor.py`, criar um dicionário vazio: `clientes_online = {}`.
 2. Dentro do `try...finally` da função `lidar_com_cliente`, garantir que, se um erro de conexão ocorrer, o cliente seja removido deste dicionário.

✓ Issue 9: crachá de entrada

- **Descrição:** Implementar o fluxo cliente-servidor para a solicitação de conexão `AUTH_CONN`, onde o cliente envia seu nickname logo após abrir o socket.
- **Tarefas:**
 1. No `cliente.py`, logo após o `connect()`, pedir ao usuário para digitar seu nome no terminal (ex: `input("Digite seu nickname: ")`).
 2. Enviar a mensagem formatada para o servidor usando a estrutura definida no protocolo: `AUTH_CONN {nickname}`.
 3. No servidor, ao receber a primeira mensagem de um cliente recém-conectado, extrair o payload e registrar no dicionário: `clientes_online[conexao] = payload`.
 4. Fazer o servidor responder com uma mensagem de confirmação (ex: `AUTH_REPLY OK`).
 - a. O servidor exibe no terminal: `[LOGIN] Usuário 'Sabrina' entrou no lobby.`
 - b. Se o cliente fechar o programa, o `finally` o remove do dicionário e o servidor exibe: `[LOGOUT] Usuário 'Sabrina' saiu.`

✓ ~~Issue 10: broadcast com~~ `SYNC_STATUS`

- **Descrição:** Sempre que alguém entra ou sai do lobby, o servidor deve avisar a **todos** os outros clientes conectados para que eles atualizem suas listas de usuários online (ação de conectar/desconectar descrita na tabela do protocolo).
- **Tarefas:**
 1. No servidor, criar uma função `fazer_broadcast(mensagem: bytes, remetente_ignorado=None)`.
 2. Essa função deve iterar sobre as chaves do dicionário `clientes_online` (que são os sockets) e executar `socket_destino.sendall(mensagem)`.
 3. Se `remetente_ignorado` for fornecido, pular o envio para ele mesmo.
 4. Integrar o `SYNC_STATUS`: logo após registrar um novo cliente na Issue 9, formatar a mensagem `SYNC_STATUS {nickname}_entrou` e usar a função de broadcast. Fazer o mesmo no bloco de desconexão (`SYNC_STATUS {nickname}_saiu`).

Fase 3: módulo cliente concorrente (sem interface gráfica do usuário)

Objetivo: Garantir que o programa cliente consiga, ao mesmo tempo, ouvir o que o servidor fala e aceitar comandos do usuário (via terminal, inicialmente) sem travar.

✓ ~~Issue 11: dilema dos bloqueios (isolando a recepção)~~

- **Descrição:** Em Python, funções como `input()` (aguarda você digitar) e `recv()` (aguarda a rede mandar dados) são chamadas de "bloqueantes". Se você colocar um depois do outro, o programa trava. O cliente precisa de uma função dedicada apenas para escutar o servidor, que rodará separada do resto do código para prevenir bloqueios na tela.

- **Tarefas:**

1. No arquivo do cliente, crie uma função chamada `escutar_servidor(client_socket)`.
2. Dentro dessa função, crie um laço contínuo `while True:`.
3. Coloque a lógica de receber dados: `dados = client_socket.recv(1024)`.
4. Decodifique os dados recebidos e imprima no terminal.
5. Adicione um bloco `try...except` e uma verificação `if not dados:` para encerrar a função caso a conexão caia.

✓ **Issue 12: ouvido em segundo plano**

- **Descrição:** Agora que temos a função que escuta infinitamente, precisamos colocá-la para rodar em paralelo usando múltiplas threads.

- **Tarefas:**

1. No fluxo principal do cliente, logo após fazer o `connect()` e enviar a mensagem inicial de `AUTH_CONN`, instancie uma thread: `thread_escuta = threading.Thread(target=escutar_servidor, args=(client_socket,))`.
2. Configure a thread como um processo em segundo plano (daemon) antes de iniciá-la: `thread_escuta.daemon = True`. Isso garante que, se você fechar o programa principal, essa thread morra junto e não fique presa rodando invisível no seu computador.
3. Inicie a thread com `thread_escuta.start()`.

✓ **Issue 13: boca do cliente (thread principal e `input`)**

- **Descrição:** A thread principal (o fluxo normal do seu script) ficará dedicada exclusivamente a ler o que você digita no terminal e enviar para o servidor, imitando a ação que os botões da futura interface gráfica farão.

- **Tarefas:**

1. Logo após o `thread_escuta.start()`, crie outro laço `while True:` no corpo principal do script.
2. Use `texto = input()` para travar essa thread esperando você digitar algo.
3. Quando o usuário digitar uma mensagem, formate-a manualmente (por exemplo, simulando o comando de chat: `f"SEND_CHAT {texto}\n"`) e codifique para bytes com `.encode('utf-8')`.
4. Envie para o servidor usando `client_socket.sendall()`.

✓ **Issue 14: encerrando a conexão limpamente no cliente**

- **Descrição:** Se o servidor cair, a `thread_escuta` vai perceber isso primeiro (o `recv` retornará vazio). Precisamos garantir que o programa inteiro saiba que a rede caiu e encerre de forma limpa.

- **Tarefas:**

1. Na função `escutar_servidor`, quando a conexão for perdida, use `print("\nConexão com o servidor encerrada.")`.
2. Adicione o comando `os._exit(0)` (após importar a biblioteca nativa `os`) para forçar o fechamento completo do script Python, derrubando o `input()` que estava travado aguardando texto.

Fase 4: sincronismo global e chat (TCP)

Objetivo: Implementar o chat global (SEND_CHAT) e o sincronismo do vestiário (SYNC_STATUS). O servidor recebe a mensagem/alteração de um cliente e faz o *broadcast* (espalha) para todos os outros.

✓ Issue 15: roteador de mensagens

- **Descrição:** Na praça de alimentação virtual, o servidor atua como um carteiro central. Quando um cliente envia uma mensagem de texto, o servidor não a guarda para si; ele deve identificar de quem veio e retransmitir para todos os outros usuários da sala.
- **Tarefas:**
 1. No `servidor.py`, dentro do laço de recepção da função que atende o cliente, adicione um `if comando == "SEND_CHAT":`.
 2. Construa a mensagem final que será repassada, juntando o nome do remetente (buscando no dicionário `clientes_online`) com o texto recebido. Exemplo: `payload_final = f"{nome_remetente}: {payload}"`.
 3. Formate a mensagem de volta para o padrão do protocolo: `SEND_CHAT {payload_final}\n`.
 4. Chame a função `fazer_broadcast` (criada na Issue 10), passando essa mensagem em bytes e informando o socket do remetente no parâmetro `remetente_ignorado` (para que o usuário não receba de volta a própria mensagem).

✓ Issue 16: vestiário distribuído

- **Descrição:** O vestiário permite alterar atributos do avatar (como a cor do avental). Essa mudança de estado é apenas uma variação da lógica do chat: o cliente avisa o servidor, e o servidor avisa o resto do mundo.
- **Tarefas:**
 1. No `cliente.py`, crie um atalho de terminal para simular o vestiário (ex: se o usuário digitar `/cor azul`, o cliente não envia isso como chat, mas formata como `SYNC_STATUS azul\n`).
 2. Envie essa mensagem via socket TCP para o servidor.
 3. No `servidor.py`, adicione um `elif comando == "SYNC_STATUS":`.
 4. Formate a mensagem para incluir quem mudou de cor: `SYNC_STATUS {nome_remetente} mudou para {payload}\n` e use a função `fazer_broadcast`.

✓ Issue 17: criptografia básica do chat

- **Descrição:** O diagrama do projeto exige que a mensagem da praça de alimentação sofra a aplicação de uma criptografia básica antes de ser enviada (`Envio SEND_CHAT via TCP`). Como a restrição é de não esconder abstrações sob bibliotecas de alto nível, implementaremos uma Cifra de César simples ou manipulação de bytes (XOR).
- **Tarefas:**
 1. Crie um arquivo utilitário `seguranca.py` .
 2. Implemente uma função `criptografar(texto_limpo, chave_deslocamento=3)` que percorre cada caractere do texto, converte para o valor numérico (usando a função nativa `ord()`), soma o deslocamento, e volta para caractere (`chr()`).
 3. Implemente a função inversa `descriptografar(texto_cifrado, chave_deslocamento=3)` .
 4. No `cliente.py` , aplique `criptografar()` no texto **antes** de colocá-lo no payload do `SEND_CHAT` .
 5. Na função `escutar_servidor` do cliente receptor, aplique `descriptografar()` no payload do `SEND_CHAT` **antes** de exibi-lo na tela.

✓ Issue 18: intérprete-do-cliente

- **Descrição:** Até a Fase 3, o cliente apenas imprimia cegamente o que o servidor mandava. Agora, a `thread_escuta` recebe `SYNC_STATUS` , `SEND_CHAT` e outros comandos. O cliente precisa processar cada pacote da forma correta.
- **Tarefas:**
 1. Na função `escutar_servidor` do `cliente.py` , utilize a função utilitária `interpretar_mensagem` (da Issue 4) na string decodificada.
 2. Crie uma estrutura condicional: se o comando for `SEND_CHAT` , descriptografe e exiba com a formatação `[CHAT]` .
 3. Se o comando for `SYNC_STATUS` , exiba com a formatação `[SISTEMA]` .

Fase 5: confiabilidade e resiliência (heartbeat)

Objetivo: Implementar o DEAD_TRIG (keep alive). O servidor deve monitorar se os clientes estão vivos e removê-los graciosamente se eles sumirem.

✓ Issue 19: pulso-do-cliente (thread de envio do `DEAD_TRIG`)

- **Descrição:** Em redes reais, se o computador de um usuário travar ou a energia cair subitamente, o TCP não consegue enviar o pacote de "fechamento de conexão" (FIN). Para o servidor, a conexão parece estar aberta ("conexão fantasma"). O diagrama do projeto resolve isso exigindo o envio contínuo da mensagem de *keep alive* (`DEAD_TRIG`) em uma thread em background.
- **Tarefas:**
 1. No `cliente.py` , importe a biblioteca nativa `time` .
 2. Crie a função `enviar_heartbeat(client_socket)` com um laço `while True:` .

3. Dentro do laço, formate e envie a mensagem `DEAD_TRIG\n` ao servidor.
4. Logo após o envio, pause a execução dessa função usando `time.sleep(5)` (para pulsar a cada 5 segundos).
5. No bloco principal, após o login, instancie e inicie uma nova thread apontando para essa função, marcando-a como `daemon = True` para que não impeça o fechamento do programa.

✓ ~~Issue 20: prontuário do servidor~~

- **Descrição:** O servidor precisa saber a que horas cada cliente deu sinal de vida pela última vez para poder julgá-los como ativos ou inativos.
- **Tarefas:**
 1. No `servidor.py`, importe a biblioteca `time`.
 2. Atualize o dicionário global `clientes_online`. Em vez de guardar apenas o nome (ex: `clientes_online[conexao] = "Sabrina"`), passe a guardar um dicionário interno com o status: `clientes_online[conexao] = {"nome": payload, "ultimo_sinal": time.time()}`.
 3. Na função que atende o cliente, adicione um `elif comando == "DEAD_TRIG":`.
 4. O servidor não deve repassar essa mensagem para os outros. Ele deve apenas atualizar o campo `"ultimo_sinal"` daquele cliente para o `time.time()` atual.
 5. Certifique-se de que qualquer outro comando (como `SEND_CHAT`) também atualize esse relógio, afinal, se o cliente mandou chat, ele está vivo.

✓ ~~Issue 21: ceifador (thread de limpeza de inativos no servidor)~~

- **Descrição:** Agora que temos os registros de horário atualizados, precisamos de um inspetor no servidor que verifique periodicamente quem parou de mandar o pulso.
- **Tarefas:**
 1. No servidor, crie uma função `monitorar_inativos()`.
 2. Em um `while True:`, adicione `time.sleep(10)` para que o servidor faça a varredura a cada 10 segundos.
 3. Itere sobre uma cópia das chaves do dicionário `clientes_online` (usando `list(clientes_online.keys())` para evitar erros de modificação durante o loop).
 4. Se o tempo atual (`time.time()`) menos o `"ultimo_sinal"` do cliente for maior que 15 segundos (o limite de tolerância):
 - Imprima `[SISTEMA] Removendo cliente inativo por timeout.`
 - Feche o socket forçadamente com `conexao.close()`.
 - Remova-o do dicionário.
 - Envie um `SYNC_STATUS` para os sobreviventes avisando que o usuário desconectou.
 5. Inicie essa função como uma thread no bloco principal do `servidor.py`, antes do laço `accept()`.

Fase 6: cozinha (matchmaking TCP)

Objetivo: Implementar o convite do jogo. Cliente A pede para jogar com Cliente B via servidor (REQ_MATCH). O servidor troca as informações de IP e porta UDP entre eles.

✓ ~~Issue 22: preparar a cozinha local (criação do socket UDP no cliente)~~

- **Descrição:** Antes de o cliente poder pedir ao servidor para iniciar um jogo, ele tem de garantir que possui uma porta preparada para receber pacotes rápidos do minijogo. Ao contrário do TCP, que usa o servidor, o UDP funcionará de forma direta (peer-to-peer).
- **Tarefas:**
 1. No ficheiro do cliente, importar a biblioteca `socket` e instanciar um novo socket configurado para UDP: `socket_udp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)`.
 2. Fazer o `bind` deste socket a uma porta local (podes usar `0` para que o sistema operativo escolha uma porta livre aleatoriamente: `socket_udp.bind((HOST, 0))`).
 3. Recuperar a porta escolhida com `porta_udp_local = socket_udp.getsockname()[1]`.
 4. Atualizar a Issue 9: quando o cliente enviar o comando `AUTH_CONN`, deve enviar também esta porta UDP no payload (exemplo: `AUTH_CONN {nickname} {porta_udp_local}`). No servidor, guarda esta porta no dicionário `clientes_online`.

✓ ~~Issue 23: lançar o desafio (comando REQ_MATCH)~~

- **Descrição:** Com os jogadores a visualizar a lista de pessoas online no lobby, o Cliente A decide desafiar o Cliente B (por exemplo, a tua colega Manuela). O cliente envia este pedido ao servidor central via TCP.
- **Tarefas:**
 1. No cliente, criar um comando de terminal para simular o clique no botão "desafiar oponente" do diagrama (exemplo: `/jogar Manuela`).
 2. Formatar a mensagem de rede com a estrutura do protocolo: `REQ_MATCH {nickname_oponente}\n`.
 3. Enviar este comando de texto pelo socket TCP que já está estabelecido com o servidor central.

✓ ~~Issue 24: o intermediário (processamento do pedido no servidor)~~

- **Descrição:** O servidor atua como um serviço de diretório. Ao receber o `REQ_MATCH`, vai procurar o IP e a porta UDP do oponente solicitado e devolvê-los ao solicitante.
- **Tarefas:**
 1. Na *thread* do servidor dedicada ao Cliente A, adicionar a condição `elif comando == "REQ_MATCH":`.
 2. Extrair o `nickname_oponente` do *payload* e verificar se essa chave existe no dicionário `clientes_online`.
 3. Se existir, extrair o IP (que o servidor obteve aquando do `accept()` do TCP) e a `porta_udp` (armazenada na Issue 22).

4. Formatar uma mensagem de resposta com estes dados. Como o protocolo precisa de crescer de forma dinâmica, cria uma nova mensagem de controlo que não estava explicitamente na tabela, mas que é vital para o diagrama: `MATCH_INFO {ip_oponente} {porta_udp_oponente}\n`.
5. Enviar de volta ao Cliente A via TCP.

Fase 7: cozinha (P2P UDP)

Objetivo: Criar a conexão direta entre dois clientes. Eles estabelecem o handshake UDP e começam a trocar mensagens de jogo (GAME_ACTION) diretamente, sem passar pelo servidor central.

✓ Issue 25: ouvido da cozinha (thread de escuta UDP)

- **Descrição:** Ao contrário do TCP, o UDP não tem o conceito de "aceitar uma ligação" (`accept()`). Os dados simplesmente "caem" na porta a qualquer momento. Portanto, precisamos de uma *thread* separada no ficheiro do cliente para ficar de escuta nesses pacotes, sem encravar o resto do programa que já gere o TCP.
- **Tarefas:**
 1. No cliente, criar uma função `escutar_p2p(socket_udp)`.
 2. Implementar um ciclo `while True:` com o comando bloqueante `dados, endereco_origem = socket_udp.recvfrom(1024)`. A diferença mágica aqui é que o `recvfrom` te devolve não só a mensagem, mas também o IP e a porta exatos de quem atirou o pacote.
 3. Decodificar a mensagem e imprimi-la no ecrã.
 4. Iniciar esta função como uma nova *thread* secundária (daemon) logo a seguir à criação do *socket* UDP da Issue 22.

✓ Issue 26: quebrar o gelo (handshake P2P)

- **Descrição:** Embora o UDP não exija ligação, na prática, os *routers* e as barreiras de proteção (*firewalls*) precisam de ver o Cliente A a enviar um pacote para o Cliente B para abrirem um "buraco" de comunicação. É para isso que serve o envio do comando `AUTH_CONN` entre os *peers*.
- **Tarefas:**
 1. No cliente, assim que receber a mensagem `MATCH_INFO` com os dados do oponente (Issue 24), extrair o IP e a porta UDP de destino.
 2. Formatar a mensagem de apresentação: `AUTH_CONN {meu_nickname} {nickname_oponente}\n`.
 3. Enviar a mensagem usando `socket_udp.sendto(mensagem_bytes, (ip_oponente, porta_udp_oponente))`.
 4. Na função `escutar_p2p`, adicionar uma condição: se a mensagem recebida for `AUTH_CONN`, registar que o *handshake* foi bem-sucedido e iniciar o estado da partida.

✓ Issue 27: ritmo acelerado (trocando `GAME_ACTION`)

- **Descrição:** Começou o jogo! Agora a velocidade importa para montar a pizza correta. Sempre que houver uma interação (por exemplo, se o teu avatar de coruja apanhar um ingrediente ou alterar as coordenadas dos *sprites* em *pixel art* na interface), precisas de avisar o outro lado instantaneamente usando `GAME_ACTION`.
- **Tarefas:**
 1. Criar um atalho de simulação no terminal (ex: `/acao queijo`), que seja processado na *thread* principal.
 2. Formatar a mensagem como `GAME_ACTION {tipo_de_acao}\n` e atirá-la com o `sendto()` para o IP e porta do adversário.
 3. No `escutar_p2p` do recetor, adicionar o `elif comando == "GAME_ACTION":` e exibir a jogada no terminal.

✓ Issue 28: Limpeza de Bancada (Fim da Partida)

- **Descrição:** Quando o minijogo termina, os *peers* têm de parar de trocar pacotes UDP e regressar em pleno ao estado de repouso no *Lobby* TCP principal.
- **Tarefas:**
 1. Implementar um comando de fim (ex: `GAME_OVER`).
 2. Quando recebido, a aplicação limpa as variáveis que guardavam o IP e a porta do oponente atual.
 3. Avisar o servidor central via TCP que a partida acabou (através de uma mensagem `SYNC_STATUS`, por exemplo), para que o servidor os volte a colocar disponíveis para outros desafios.

Fase 8: integração gráfica (GUI)

Objetivo: Desenhar o menu básico (CustomTkinter/Pygame) e conectar os botões e caixas de texto à lógica de rede que já foi exaustivamente testada no terminal.

✓ Issue 29: a ponte entre a rede e a interface (eventos e callbacks)

- **Descrição:** A biblioteca gráfica (como o CustomTkinter) possui o seu próprio ciclo contínuo, chamado `mainloop()`, que atualiza o ecrã a 60 *frames* por segundo. Se a interface gráfica ficar encravada à espera de uma mensagem da rede, a janela "congela" e o sistema operativo sugere fechar o programa. Precisamos de conectar o ficheiro `interface.py` ao `rede.py` através de filas de mensagens (*Queues*) ou funções de *callback*.
- **Tarefas:**
 1. No ficheiro principal `main.py`, importa a biblioteca gráfica escolhida. O CustomTkinter é excelente para estruturar os formulários, botões e a caixa de *chat* do *Lobby*.
 2. Modifica a `thread_escuta` (Issue 12) para que, ao invés de usar `print()` para o terminal, ela coloque as mensagens recebidas numa fila segura (`queue.Queue`) nativa do Python.
 3. Na interface gráfica, cria uma função atualizadora que usa o método `.after(100, verificar_fila)` do CustomTkinter para inspecionar essa fila a cada 100 milissegundos

e atualizar os textos no ecrã sem travar o rato.

✓ ~~Issue 30: ecrã de login (substituindo o terminal)~~

- **Descrição:** O primeiro contacto do jogador com a aplicação é o ecrã de conexão. Precisamos de capturar o nome introduzido e desencadear o fluxo TCP.
- **Tarefas:**
 1. Desenhar uma janela com um campo de entrada de texto (`CTkEntry`) e um botão "Entrar" (`CTkButton`).
 2. Ligar o clique do botão à função que executa o `connect()` ao `127.0.0.1:5050` e envia o pacote `AUTH_CONN {nickname_digitado}` .
 3. Mostrar um indicador de carregamento ("A aguardar servidor...").
 4. Se a ligação falhar, apresentar uma caixa de diálogo de erro. Se for um sucesso, destruir o ecrã de *login* e abrir o ecrã do *Lobby* principal.

✓ ~~Issue 31: praça de alimentação (chat e contatos em tempo real)~~

- **Descrição:** A interface central onde o utilizador vê o *chat* global criptografado e a lista de jogadores disponíveis para o pareamento.
- **Tarefas:**
 1. Dividir a janela do *Lobby* em dois blocos: um painel principal rolável para o *chat* e uma barra lateral para a lista de jogadores online.
 2. Criar uma caixa de texto e um botão "Enviar". Quando clicado, encripta o texto e envia o `SEND_CHAT` via TCP.
 3. Quando a `verificar_fila` (Issue 29) detetar uma mensagem de *chat* decifrada, adicionar esse texto ao painel rolável.
 4. Quando detetar um `SYNC_STATUS` de alguém a conectar-se ou desconectar-se, limpar a barra lateral e redesenhar a lista de nomes online atualizada.

✓ ~~Issue 32: o vestiário visual~~

- **Descrição:** Permitir a personalização do avatar através de botões na interface em vez de comandos de texto ocultos, propagando a mudança visualmente para todos.
- **Tarefas:**
 1. Adicionar um botão ou menu suspenso no *Lobby* para selecionar a cor do avental ou tipo de roupa.
 2. Ao selecionar uma opção, disparar o envio do pacote `SYNC_STATUS` para o servidor.
 3. Quando a interface receber um `SYNC_STATUS` informando a mudança de atributo de um jogador, alterar a cor ou o ícone do avatar junto ao nome dele na lista de jogadores online.

✓ ~~Issue 33.1: convite e estado de ocupação (refatoração TCP)~~

- **Descrição:** Garantir que os jogadores podem aceitar ou recusar partidas, e impedir que jogadores já ocupados na cozinha sejam desafiados.
- **Tarefas:**
 1. No servidor (`servidor.py`), adicionar um campo `"status": "disponivel"` no dicionário `clientes_online` para cada jogador.
 2. Quando o Cliente A envia `REQ_MATCH`, o servidor verifica se o status do B é "disponivel". Se não for, devolve um erro ao Cliente A (`MATCH_REJECT Ocupado`).
 3. Se estiver disponível, o servidor não devolve o IP de imediato; em vez disso, roteia o pedido para o Cliente B: `CHALLENGE_INVITE {nome_do_A}` .
 4. O Cliente B exibe um *pop-up* na interface gráfica: "Aceitar desafio?". Se clicar sim, envia `ACCEPT_MATCH` .
 5. Só então o servidor altera o status de ambos para `"jogando"` , faz um *broadcast* de `SYNC_STATUS` para o *Lobby* saber que eles estão indisponíveis, e envia os IPs e portas UDP para os dois iniciarem o *handshake* P2P.
- **Critérios de aceite:**
 - Jogadores em partida ficam com um cadeado ou ícone vermelho no lobby e não podem receber novos convites.
- ☐ Issue 33.2: renderização da cozinha e temporizador (Pygame)
- **Descrição:** Criar a base visual do minijogo de montar pizzas, o relógio de 1 minuto e a mecânica das 3 camadas.
- **Tarefas:**
 1. Criar um ficheiro `cozinha.py` que inicia uma janela Pygame isolada.
 2. Carregar as imagens (*sprites*) dos ingredientes (ex: massa base, molho, queijo, pepperoni).
 3. Desenhar a interface: área de montagem da pizza ao centro, pontuação da Rafaela (esquerda), pontuação da Manuela (direita) e um relógio gigante no topo (60 segundos).
 4. Implementar a mecânica de cliques ou teclas (ex: pressionar '1' para molho, '2' para queijo, '3' para pepperoni), desenhando as imagens sobrepostas na tela.
- **Critérios de aceite:**
 - O jogador consegue empilhar os ingredientes visualmente. Se a pizza tiver 3 ingredientes, o jogo detecta que a montagem daquele pedido foi finalizada.
- ☐ Issue 33.3: lógica central (a fila de pedidos)
- **Descrição:** Garantir que ambos os jogadores recebem a mesma sequência de clientes virtuais para que a disputa seja justa.
- **Tarefas:**

1. Definir que quem enviou o desafio original atuará como "Host" da geração de pedidos.
 2. O "Host" gera uma lista de 20 pedidos aleatórios (cada pedido sendo um *array* de 3 ingredientes).
 3. Assim que o *handshake* UDP (Issue 26) é feito, o "Host" envia a lista de pedidos serializada numa string UDP para o outro jogador.
 4. Ambos os clientes leem a lista. O primeiro pedido aparece no papelzinho.
 5. Se a pizza montada na Issue 33.2 coincidir exatamente com os 3 itens do papelzinho, o jogador soma 10 reais, o papelzinho avança para o próximo pedido da lista, e a área de montagem é limpa.
- **CrITÉrios de aceite:**
 - Ambos as telas mostram exatamente a mesma pizza no papelzinho de pedido número 1, número 2, etc.
- ☐ Issue 33.4: sincronismo de competição P2P (troca de pontuação UDP)
- **Descrição:** Manter a pressão do jogo trocando dados em tempo real pela conexão UDP.
 - **Tarefas:**
 1. Sempre que um jogador concluir uma pizza corretamente (Issue 33.3), enviar um pacote UDP para o adversário: `GAME_ACTION SCORE {minha_pontuação}`.
 2. Na *thread* que escuta o UDP, ao receber este pacote, atualizar imediatamente o texto da pontuação do adversário na tela do Pygame.
 3. Quando o temporizador de 60 segundos chegar a zero, travar os comandos de teclado.
 4. Comparar os 10 reais locais com os reais recebidos do oponente. Apresentar o ecrã final: "Vitória!", "Derrota" ou "Empate".
 5. Fechar o *socket* UDP P2P e regressar ao ecrã do *Lobby*, enviando o comando ao servidor TCP para voltar ao *status* `"disponivel"`.
 - **CrITÉrios de aceite:**
 - Ao fazer uma pizza, o meu oponente vê a minha pontuação subir instantaneamente. Quando o relógio bate nos 0 segundos, o jogo acaba de forma sincronizada, declara o vencedor pelo dinheiro arrecadado e devolve os jogadores ao lobby.